

Deep learning

10

In this chapter, you will learn deep learning, a powerful family of techniques based on *artificial neural networks* with broad applications in computer vision, natural language processing, machine translation, social network analysis, and so on. It has been used in a variety of data mining tasks, including classification, clustering, outlier detection, and reinforcement learning. We start with the basic concepts (Section 10.1). Then, we introduce key algorithmic techniques for training an effective deep learning model (Section 10.2), and commonly used deep learning model architectures, including convolutional neural networks (Section 10.3), recurrent neural networks (Section 10.4), and graph neural networks (Section 10.5).

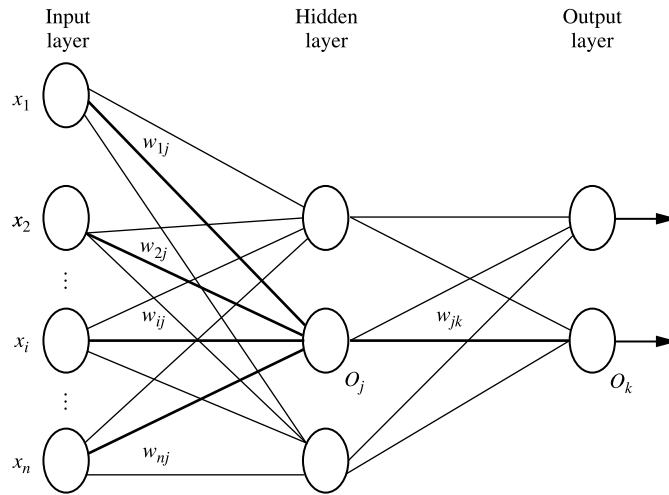
10.1 Basic concepts

10.1.1 What is deep learning?

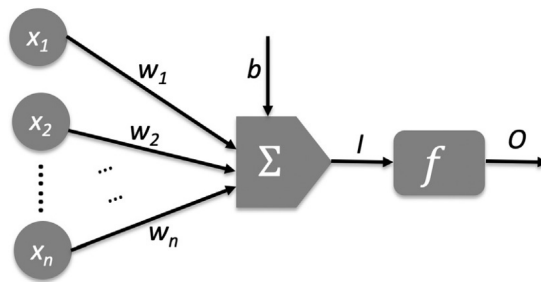
Deep learning is based on **artificial neural networks** (ANNs) (or **neural networks** for short). Fig. 10.1 shows an illustrative example of neural network, and we will dive into details in a minute. Roughly speaking, a **neural network** is a set of connected input-output units in which each connection has a weight associated with it. During the learning phase, the network learns by adjusting the weights in order to predict the correct target values (e.g., class labels) of the input tuples.

A neural network is made up of interconnected **units**. So, what is a unit? Actually, we have already seen it! Recall that in Chapter 6, we introduced some basic classifiers, such as perceptron and logistic regression. Consider a data tuple X with n attributes: $X = (X_1, X_2, \dots, X_n)$. Both perceptron and logistic regression classifier first take a linear weighted sum of different attributes or features $\sum_{i=1}^n X_i w_i + b$, where w_i ($i = 1, \dots, n$) are the weights and b is the bias scalar. Then, perceptron predicts the class label based on the *sign* of the linear weighted sum $\hat{y} = \text{sign}(\sum_{i=1}^n X_i w_i + b)$, where $\text{sign}(z) = 1$ if $z \geq 0$ and $\text{sign}(z) = 0$ otherwise; logistic regression predicts the class posterior probability based on the sigmoid function of the linear weighted sum $P(y = 1|X) = \sigma(\sum_{i=1}^n X_i w_i + b)$, where $\sigma(z) = \frac{1}{1+\exp(-z)}$ is the sigmoid function. In the neural network terminology, both perceptron and logistic regression can be viewed as a unit.

Formally, a unit is a mathematical function that (1) takes a linear weighted sum of the input, and then (2) passes the sum through an **activation function** $f(\cdot)$. The activation function $f(\cdot)$ is typically a nonlinear function, such as the sign function in perceptron and the sigmoid function $\sigma(\cdot)$ in logistic regression. Many other choices for the activation function exist, and we will introduce some of them in Section 10.2. For the deep learning algorithms, it is convenient to introduce two additional notations. The first one is the net input $I = \sum_{i=1}^n X_i w_i + b$ of the activation function. The second is the output of the unit $O = f(I)$. A pictorial illustration of a unit is presented in Fig. 10.2.

**FIGURE 10.1**

Multilayer feed-forward neural network.

**FIGURE 10.2**

An illustrative example of unit. Given the inputs $X_1, \dots, X_n$, a unit takes a linear weighted sum and then passes the sum through an activation function $f(\cdot)$. $I = \sum_{i=1}^n X_i w_i + b$ is the net input of the activation function and $O = f(I)$ is the output of the unit. w_i ($i = 1, \dots, n$) are weights and b is the bias scalar.

A neural network is essentially a collection of interconnected units. Depending on how different units are organized with each other, there are many different kinds of neural networks. Among them, a very important and powerful type of neural network is called **multilayer feed-forward neural network**. Formally, a multilayer feed-forward neural network consists of an *input layer*, one or more *hidden layers*, and an *output layer*. An example of a multilayer feed-forward network is shown in Fig. 10.1.

Each layer is made up of units. The inputs to the network correspond to the attributes measured for each training tuple. The inputs are fed simultaneously into the units making up the **input layer**. These

inputs pass through the input layer and are then weighted and fed simultaneously to a second layer of “neuronlike” units, known as a **hidden layer**. The outputs of the hidden layer units can be input to another hidden layer, and so on. The number of hidden layers could be arbitrary (1 hidden layer in this example), outputs of the last hidden layer are input to units making up the **output layer**, which emits the network’s predictions for given tuples.

The units in the input layer are called **input units**. The units in the hidden layers and output layer are sometimes referred to as **neuroids** or **neurons**, due to their symbolic biological basis, or as **output units**. The multilayer feed-forward neural network shown in Fig. 10.1 has two layers of output units. Therefore we say that it is a **two-layer** neural network. Notice that the input layer is not counted because it serves only to pass the input values to the next layer. In other words, the activation function for an input unit is always an *identity function*: $O = f(I) = I$. Similarly, a network containing two hidden layers is called a *three-layer* neural network, and so on. It is a feed-forward network since none of the weights cycles back to an input unit or to a previous layer’s output unit. It is **fully connected** in the sense that each unit provides input to each unit in the next forward layer.

Each output unit takes, as input, a weighted sum of the outputs from units in the previous layer (see Fig. 10.1). It applies a nonlinear activation function to the weighted sum of the inputs. If we know the weights connecting to each layer and the activation function of each unit, we can feed any data tuple into the input layer of the neural network and calculate its output from the output layer, which can be used for classification, clustering, and so on. Multilayer feed-forward neural networks are able to model the class prediction as a nonlinear combination of the inputs. From a statistical point of view, they perform nonlinear regression. Multilayer feed-forward networks with sufficient depth and width, given enough hidden units and enough training samples, can closely approximate any function. In a multilayer feed-forward network, the output of a hidden unit is used as the input to units in the next layer. This gives neural network an incredible ability to learn more complicated, often semantically more meaningful features from the simpler ones. Put it in another way, from left to right of Fig. 10.1, the outputs of units at each layer of a multilayer feed-forward neural networks form a hierarchy of learned features at increasingly more complex levels.

Compared with other data mining methods we have seen before, such as logistic regression and Support Vector Machines (SVMs), being able to learn *any* nonlinear function that maps the input data tuple to the output (e.g., the class label) and the automatic feature learning are two major advantages of neural networks. Let us use the classic XOR problem in Fig. 10.3 to further illustrate this. In Fig. 10.3(a), there are four training tuples, each of which is represented by two attributes X_1 and X_2 . Two positive training tuples are at (0, 1) and (1, 0), respectively, and two negative training tuples are at (0, 0) and (1, 1), respectively. This training set is linearly inseparable, meaning that there is no linear classifier (e.g., perceptron) that is able to separate the two positive tuples from the two negative ones. However, we can use two perceptrons (the two dashed lines in Fig. 10.3(a)) to separate the positive tuples from the negative tuples. The first perceptron is in the form of $O_1 = \text{sign}(X_2 - X_1 - 0.5)$. The second perceptron is in the form of $O_2 = \text{sign}(X_2 - X_1 + 0.5)$. By combining these two perceptrons (each of which is a linear classifier) together, we have a nonlinear classifier that perfectly separates the two positive tuples from the two negative tuples; that is, if $X_2 - X_1 - 0.5 \geq 0$ or $X_2 - X_1 + 0.5 < 0$, predict it as a positive tuple; otherwise, predict it as a negative tuple.

This nonlinear classifier can be implemented as a two-layer feed-forward neural network in Fig. 10.3(b) marked by the weights and scalars of each unit. There are two units (U_3 and U_4) in the hidden layer, each of which takes the output of U_1 and U_2 (i.e., X_1 and X_2) to build a per-

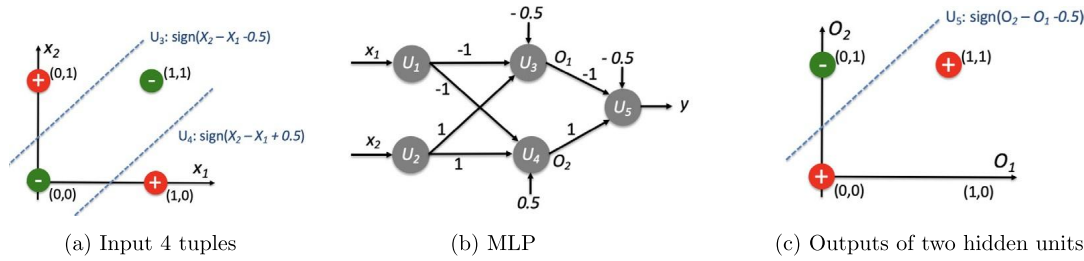


FIGURE 10.3

Solving XOR problem with a two-layer feed-forward neural network with *sign* as the activation function, which is also called multilayer perceptron (MLP). Note that in (c), both negative tuples share the same representation (O_1, O_2), both being at (0, 1).

ceptron. In particular, the unit U_3 corresponds to the first perceptron shown in Fig. 10.3(a): $O_1 = \text{sign}(X_2 - X_1 - 0.5)$; and the unit U_4 corresponds to the second perceptron shown in Fig. 10.3(a): $O_2 = \text{sign}(X_2 - X_1 + 0.5)$. In the output layer, there is only one unit U_5 , which takes the outputs of the two hidden units (O_1 and O_2) to build another perceptron, that is, $y = \text{sign}(O_2 - O_1 - 0.5)$ (shown as the dashed line in Fig. 10.3(c)). In this way, we are able to separate the two positive tuples from the two negative ones. This neural network has three layers. The input layer has two units (U_1 and U_2), each of which just passes through the corresponding input attributes (X_1 and X_2) to the hidden layer. In this example, we use the *sign* function as the activation function for all neurons (U_3 , U_4 , and U_5), and this kind of neural network is also called multilayer perceptron (MLP).

In this example, the third perceptron at unit U_5 takes O_1 and O_2 (i.e., the outputs of unit U_3 and U_4), as its input features. In other words, both O_1 and O_2 can be viewed as the *learned* features by the two hidden units. O_1 represents whether or not the input tuple is above ($O_1 = 1$) or below ($O_1 = 0$) of the decision boundary of the first perceptron (the upper dashed line in Fig. 10.3(a)). Likewise, O_2 represents whether or not the input tuple is above ($O_2 = 1$) or below ($O_2 = 0$) of the decision boundary of the second perceptron (the lower dashed line in Fig. 10.3(a)). Compared with the original input attributes (X_1 and X_2), these two learned features are semantically more meaningful. As we can see from Fig. 10.3(c), the tuples with these two newly learned features are now linearly separable. In contrast, in the original feature space (Fig. 10.3(a)), these training tuples cannot be separated from each other by any linear classifier.

The ability of neural networks to automatically learn features naturally motivates to use neural networks with many hidden layers, namely, **deep neural networks**. Deep neural networks have an incredible capability to learn and represent features at different abstraction levels (e.g., one level at each hidden layer), where the more complicated features are learned based on the simpler ones. Let us look at two examples to elaborate on this. In computer vision, we can build a type of deep neural network called convolutional neural networks (it will be introduced in Section 10.3) to recognize different objects (e.g., car, person, animal) from the input image. The input layer of the neural network contains the raw pixels (e.g., one pixel for each input unit), which represent the feature at the lowest semantic level. The output (i.e., the learned feature from the raw pixels at the input layer) of the first hidden layer might correspond

to edges of the input image.¹ The output (i.e., the learned feature from the edges of the first hidden layer) of the second hidden layer might correspond to contours or corners of the input image. The output (i.e., the learned feature from the edges and contours of the second hidden layer) of the third hidden layer might correspond to parts of the objects (e.g., nose, car wheel) of the input image. Finally, the output layer can easily learn a classifier to recognize different objects based on the object parts at the output layer. For text mining, we can build a type of deep neural network called recurrent neural networks (it will be introduced in Section 10.4) to classify text documents into different categories. The first layer might take the raw characters as input and output the tokens (e.g., words, punctuation); the second layer might take the tokens as input and output phrases; and the final output layer might take the phrases as input and output category that the input document belongs to. While it is very difficult to directly train a classifier (e.g., logistic regression for object recognition or document categorization) based on the raw features of the input data (e.g., the raw pixels of images, the characters of documents), a deep neural network decomposes this task into a sequence of interdependent subtasks (one at each hidden or output layer), each of which learns a semantically more meaningful feature from the feature at the previous layer.

On the one hand, the core algorithmic framework to train a deep neural network, namely **backpropagation** algorithm, has largely remained the same as for the traditional feed-forward neural networks with only a few hidden layers. Therefore we will first introduce this algorithm in Section 10.1.2. On the other hand, it becomes significantly more challenging to train a deep neural network due to its increased number of layers. We will introduce such challenges in Section 10.1.3 and the solutions to handle these challenges in Section 10.2.

Deep learning is closely related to **representation learning**, which aims to automatically learn effective representation (i.e., features, attributes) from the input data to facilitate data mining tasks (e.g., classification, clustering). Notice that the scope of representation learning is broader than deep learning, in that there exist nonneural network methods to automatically learn representation from the input data. An example is principal component analysis (introduced in Chapter 2), where each principal component, a linear combination of the input attributes, can be viewed as a learned new feature.

10.1.2 Backpropagation algorithm

For the example in Fig. 10.3, the weights and bias values are given. However, how can we automatically learn such parameters, including the weight vectors and the bias scalars, from the training tuples? A foundational technique is called **backpropagation** algorithm. Backpropagation learns by iteratively processing a data set of training tuples, comparing the network's prediction for each tuple with the actual known *target* value. The target value may be the known class label of the training tuple (for classification problems) or a continuous value (for numeric prediction). For each training tuple, the weights as well as the bias values are modified so as to minimize the difference or the disagreement (e.g., mean-squared loss) between the network's prediction and the actual target value. These modifications are made in the “backward” direction (i.e., from the output layer) through each hidden layer down to the first hidden layer (hence the name *backpropagation*). Although it is not guaranteed, in general, the weights will often eventually converge, when the learning process stops. The algorithm is summarized

¹ In computer vision, an edge is where the color or the gray intensity of a group of adjacent pixels suddenly change.

Algorithm: Backpropagation. Neural network learning for classification or numeric prediction, using the backpropagation algorithm.

Input:

- D , a data set consisting of the training tuples and their associated target values;
- η , the learning rate;
- $network$, a multilayer feed-forward network.

Output: A trained neural network (i.e., the weights w_{ij} and bias b_j for each hidden or output unit).

Method:

```

(1) Initialize all weights and biases in  $network$ ;
(2) while terminating condition is not satisfied {
(3)   for each training tuple  $X$  with target output  $T$  in  $D$  {
(4)     // Propagate the inputs forward:
(5)     for each input layer unit  $j$  {
(6)        $O_j = I_j$ ; // output of an input unit is its actual input value
(7)     for each hidden or output layer unit  $j$  {
(8)        $I_j = \sum_i w_{ij} O_i + b_j$ ; // compute the net input of unit  $j$  with respect to
           the units of the previous layer,  $i$ 
(9)        $O_j = \frac{1}{1+e^{-I_j}}$ ; } // compute the output of each unit  $j$  (sigmoid activation)
(10)    // Backpropagate the errors:
(11)    for each unit  $j$  in the output layer
(12)       $\delta_j = O_j(1 - O_j)(T_j - O_j)$ ; // compute the error (mean-squared loss)
(13)    for each unit  $j$  in the hidden layers, from the last to the first hidden layer
(14)       $\delta_j = O_j(1 - O_j)(\sum_l \delta_l w_{jl})$ ; // compute the error with respect to
           the next higher layer,  $l$ 
(15)    for each weight  $w_{ij}$  in  $network$  {
(16)       $\Delta w_{ij} = \eta \delta_j O_i$ ; // weight increment
(17)       $w_{ij} = w_{ij} - \Delta w_{ij}$ ; } // weight update
(18)    for each bias  $b_j$  in  $network$  {
(19)       $\Delta b_j = \eta \delta_j$ ; // bias increment
(20)       $b_j = b_j - \Delta b_j$ ; } // bias update
(21)  } }
```

FIGURE 10.4

Backpropagation algorithm.

in Fig. 10.4. The steps involved are expressed in terms of inputs I_i , outputs O_i , and errors δ_i and may seem awkward if this is your first look at neural network learning. However, once you become familiar with the process, you will see that each step is inherently simple. The steps are described next. Note that the algorithm described in Fig. 10.4 uses the mean-squared loss and sigmoid activation function. We can generalize the algorithm in Fig. 10.4 with other types of loss functions or activation functions.

Initialize the weights: The weights in the network are initialized to small random numbers (e.g., ranging from -1.0 to 1.0 or -0.5 to 0.5). Each unit has a *bias* associated with it, which are similarly initialized to small random numbers. An important aspect for initializing the model parameters is to *break symmetry*. For the example in Fig. 10.5, both U_3 and U_4 are connected to the same input units (U_1 and U_2). Therefore it is critical to make sure that the initial weights for U_3 are different from those for U_4 , i.e., $w_{13} \neq w_{14}$ and $w_{23} \neq w_{24}$. Otherwise, U_3 and U_4 become *unidentifiable*; that is, their weights will be updated in the same way and thus will always be the same. Other than random initial-

ization, in the next section, we will introduce an effective strategy, called “pretraining,” which presets the weights and scalars in a certain way.

Each training tuple, X , has a target output T , which could be the actual class label for the classification task or the numerical value for the regression task. It is processed by the following steps. See Fig. 10.5 for a pictorial illustration.

Propagate the inputs forward: First, the training tuples are fed to the network’s input layer. The inputs pass through the input units, unchanged. That is, for an input unit, j , its output, O_j , is equal to its input value, I_j . Next, the net input and output of each unit in the hidden and output layers are computed. The net input to a unit in the hidden or output layers is computed as a linear weighted sum of its inputs. To help illustrate this point, a hidden layer or output layer unit is shown in Fig. 10.2. Each such unit has a number of inputs to it that are, in fact, the outputs of the units connected to it in the previous layer. Each connection has a weight. To compute the net input to the unit, each input connected to the unit is multiplied by its corresponding weight, and this is summed. Given a unit, j in a hidden or output layer, the net input, I_j , to unit j is

$$I_j = \sum_i w_{ij} O_i + b_j, \quad (10.1)$$

where w_{ij} is the weight of the connection from unit i in the previous layer to unit j ; O_i is the output of unit i from the previous layer; and b_j is the bias of the unit. The bias acts as a threshold to adjust the net input of the unit.²

Each unit in the hidden and output layers takes its net input and then applies an activation function to it, as illustrated in Fig. 10.2. The function symbolizes the activation of the neuron represented by the unit. Here, the sigmoid function is used. (Recall that in Chapter 6, we have used the sigmoid function to train the logistic regression classifier.) Given the net input I_j to unit j , then O_j , the output of unit j , is computed as

$$O_j = \frac{1}{1 + e^{-I_j}}. \quad (10.2)$$

This function is also referred to as a *squashing function*, because it maps a large input domain onto the smaller range of 0 to 1. The sigmoid function is nonlinear and differentiable, allowing the backpropagation algorithm to model classification problems that are linearly inseparable.

We compute the output values, O_j , for each hidden layer, up to and including the output layer, which gives the network’s prediction. In practice, it is a good idea to cache (i.e., save) the intermediate output values at each unit as they are required again later when backpropagating the error. This trick can substantially reduce the amount of computation required.

Backpropagate the error: The error is propagated backward to reflect the accuracy of the current network’s prediction, which is in turn used to update the weights and biases. For a unit j in the output layer, the error δ_j is computed by

$$\delta_j = O_j(1 - O_j)(O_j - T_j), \quad (10.3)$$

² To see this, we can conceptually think of the bias b_j as the bias scalar in a linear regression model. In other words, it tells the default (i.e., biased) value of the net input I_j , without any output from the previous layer (i.e., $O_i = 0$).

where O_j is the actual output of unit j , and T_j is the known target value of the given training tuple. Note that for some data mining tasks, such as multiclass classification, there are multiple output units (one unit for each class label), and the index j is used for different output units. For simpler tasks (e.g., binary classification, regression), there is only one unit in the output layer. In that case, the index j can be omitted. In Eq. (10.3), $O_j(1 - O_j)$ is the derivative of the sigmoid function with respect to the net input, i.e. $\frac{\partial O_j}{\partial I_j} = O_j(1 - O_j)$.

To compute the error of a hidden layer unit j , the weighted sum of the errors of the units connected to unit j in the next higher layer is considered. The error of a hidden layer unit j is

$$\delta_j = O_j(1 - O_j) \left(\sum_l \delta_l w_{jl} \right), \quad (10.4)$$

where w_{jl} is the weight of the connection from unit j to unit l in the next higher layer, δ_l is the error of unit l , and $O_j(1 - O_j)$ is the derivative of the sigmoid function with respect to the net input, i.e. $\frac{\partial O_j}{\partial I_j} = O_j(1 - O_j)$.

The weights and biases are updated to reflect the propagated errors. Weights are updated by the following equations, where Δw_{ij} is the change in weight w_{ij} .

$$\Delta w_{ij} = \eta \delta_j O_i. \quad (10.5)$$

$$w_{ij} = w_{ij} - \Delta w_{ij}. \quad (10.6)$$

“What is η in Eq. (10.5)?” η is the **learning rate**, a constant typically having a value between 0.0 and 1.0. Backpropagation uses using a gradient descent method to search for a set of weights that fits the training data with the goal of minimizing the mean-squared loss between the network’s prediction and the known target value of the tuples.³ The learning rate helps avoid getting stuck at a local minimum in the decision space (i.e., where the weights appear to converge but are not the optimum solution) and encourages finding a high-quality solution. If the learning rate is too small, then learning will occur at a very slow pace. If the learning rate is too large, then oscillation between inadequate solutions may occur. A rule of thumb is to set the learning rate to $1/t$, where t is the number of iterations through the training set so far. We will talk more about the learning rate η in Section 10.2.2.

Biases are updated by the following equations, where Δb_j is the change in bias b_j :

$$\Delta b_j = \eta \delta_j. \quad (10.7)$$

$$b_j = b_j - \Delta b_j. \quad (10.8)$$

For a given tuple X with the target value T , let $\hat{T}$ be the predicted output of X . The predicted value $\hat{T}$ is the output of the output unit, which is a (complicated) function of the weights and bias scalars. For the algorithm in Fig. 10.4, we use the mean-squared loss L to measure the disagreement between the predicted and actual target values of training tuple X : $L = \frac{1}{2}(T - \hat{T})^2$. In some applications (e.g., multiclass classification), the training tuple X has multiple target values. In this case, both T and $\hat{T}$ are vectors: $T = (T_1, T_2, \dots, T_C)$ and $\hat{T} = (\hat{T}_1, \hat{T}_2, \dots, \hat{T}_C)$. The mean-squared loss can be defined in a similar way, that is, $L = \frac{1}{2} \sum_{j=1}^C (T_j - \hat{T}_j)^2$. Mathematically, the error term δ_i in Eq. (10.3) and

³ The gradient descent method was used for training logistic regression classifier in Chapter 7.

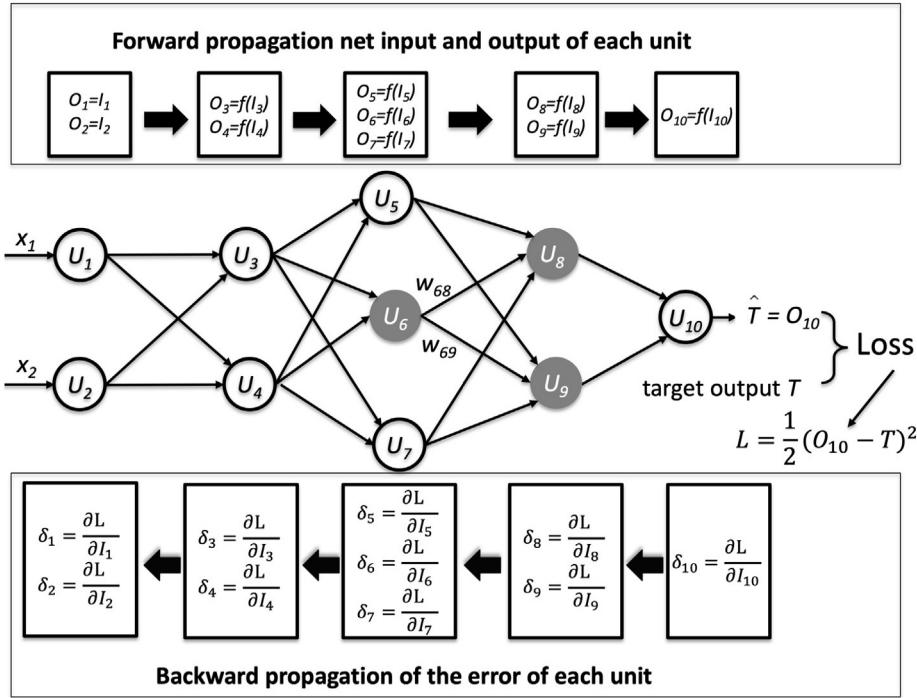
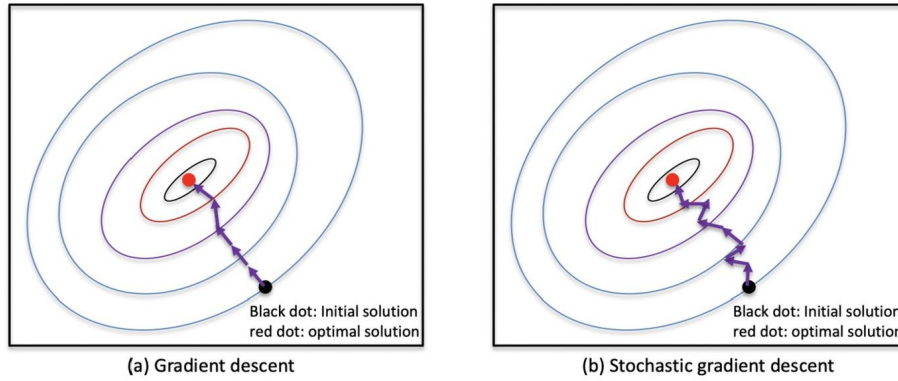


FIGURE 10.5

An illustration of the backpropagation algorithm. Given a training tuple, the algorithm first forward propagates the net input and output of each unit (the upper part of the figure). Then, it backward propagates the error of each unit (the bottom part of the figure). The output O_{10} of unit 10 gives the predicted target value $\hat{T} = O_{10}$, which is a function of the weights and bias scalars. The goal of backpropagation is to adjust the weights and bias scalars so that the predicted output matches the actual target output T as well as possible. That is, to minimize the loss $L = \frac{1}{2}(\hat{T} - T)^2$. The error of a given unit can be recursively computed based on the errors of the units it connects to in the next higher layer. For example, the error of unit 6 can be computed based on the errors of unit 8 and unit 9 (shaded in the figure). That is, $\delta_6 = O_6(1 - O_6)(w_{68}\delta_8 + w_{69}\delta_9)$. For clarity, the bias scalar or the weights between other units are not shown in the figure. Note that we actually do not need to calculate $\delta_1 = \delta_2 = 0$, since the input units (U_1 and U_2) just pass through the input attributes: $O_1 = I_1 = X_1$ and $O_2 = I_2 = X_2$.

Eq. (10.4) is the derivative of the loss L with respect to the net input of unit i : $\delta_i = \frac{\partial L}{\partial I_i}$. Accordingly, in Eq. (10.6) and Eq. (10.8), we update the weights and bias based on gradient descent. The intuition is that we want to update the weights and bias so that the loss L will decrease most. In some literature, the error term δ_i is defined as the derivative of the loss L with respect to the output of unit i : $\delta_i = \frac{\partial L}{\partial O_i}$. One can develop a similar algorithm as in Fig. 10.4 with this definition, although some mathematical details (e.g., the equations for updating the error terms) will be different. Fig. 10.5 presents an illustration of backpropagation algorithm.

Each iteration in the `while` loop of Fig. 10.4 is called an **epoch**. Note that in Fig. 10.4, we update the weights and bias values after the presentation of each tuple. Alternatively, the weight and bias

**FIGURE 10.6**

Comparison of gradient descent and stochastic gradient descent. Each ellipse is a contour of the function (e.g., the loss function in a neural network) to minimize. Gradient descent (the purple (mid gray in print version) arrows in (a)) finds the best direction to decrease the objective function value. It needs less epochs to find the optimal solution. However, in each epoch, it needs to use all the training tuples to find the best direction (i.e., the gradient). Stochastic gradient descent (the purple (mid gray in print version) arrows in (b)) finds a direction that approximates best direction to decrease the loss function value. It needs more epochs to find the optimal solution. However, in each epoch, it only needs a mini-batch of training tuples. Overall, stochastic gradient descent is often much more computationally efficient than gradient descent.

increments (Δw_{ij} and Δb_j) could be accumulated in variables, so that the weights and biases are updated after all the tuples (called **full batch**) in the training set have been presented. In theory, the mathematical derivation of backpropagation employs the latter strategy, since the gradients computed in this way give the best direction to reduce the disagreement (or loss) between the actual target values and predicted values of training tuples. In practice, we often use another strategy called **stochastic gradient descent**, which works as follows. In each epoch, we randomly and independently sample a small number of training tuples (called **mini-batch**). The weight and bias increments of each sampled training tuple are accumulated, that is, $\Delta b_j = \eta \sum_k \delta_j^k$ and $\Delta w_{ij} = \eta \sum_k \delta_j^k O_j^k$, where k is the index of sampled mini-batch. Compared with the standard gradient descent, which needs all training tuples to compute the exact gradient in one epoch, stochastic gradient descent needs more epochs to terminate, but it only needs a small number of sampled tuples to calculate an estimated gradient in each epoch. The benefit of being able to quickly estimate the gradient at each epoch often outweighs the fact that stochastic gradient descent needs more epochs. Therefore the overall running time of stochastic gradient descent is often much smaller than the standard gradient descent. See Fig. 10.6 for an illustration and comparison.

Terminating condition: Training stops when

- All Δw_{ij} in the previous epoch are so small as to be below some specified threshold, or
- The percentage of tuples misclassified in the previous epoch is below some threshold, or
- A prespecified number of epochs has expired.

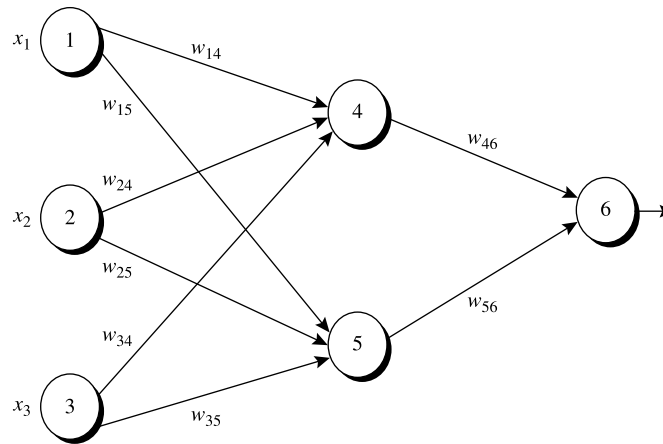


FIGURE 10.7

Example of a multilayer feed-forward neural network.

Table 10.1 Initial inputs, weights, and bias values.

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	b_4	b_5	b_6
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

In practice, several hundreds of thousands of epochs may be required before the weights will converge.

“How efficient is backpropagation?” The computational efficiency depends on the time spent training the network. Given $|D|$ tuples and w weights, each epoch requires $O(|D| \times w)$ time. However, in the worst-case scenario, the number of epochs can be exponential in n , the number of inputs. In practice, the time required for the networks to converge is highly variable. A number of techniques exist that help speed up the training time. We will introduce some key algorithmic techniques to accelerate the computation in Section 10.2.

Example 10.1. Sample calculations for learning by the backpropagation algorithm. Fig. 10.7 shows a multilayer feed-forward neural network. Let the learning rate $\eta = 0.9$. The initial weight and bias values of the network are given in Table 10.1, along with the first training tuple, $X = (1, 0, 1)$, with a class label of 1.

This example shows the calculations for backpropagation, given the first training tuple, X . The tuple is fed into the network, and the net input and output of each unit are computed. These values are shown in Table 10.2. The error of each unit is computed and propagated backward. The error values are shown in Table 10.3. The weight and bias updates are shown in Table 10.4. $\square$

“How can we classify an unknown tuple using a trained network?” To classify an unknown tuple, X , the tuple is input to the trained network, and the net input and output of each unit are computed. There is no need for computation or backpropagation of the error. If there is one output node per class, then the output node with the highest value determines the predicted class label for X . This is called **Maxout** in the sense that the maximum output value of all output units $O^i (i = 1, \dots, C)$ is used as the

Table 10.2 Net input and output calculations.

Unit, j	Net Input, I_j	Output, O_j
4	$0.2 + 0 - 0.5 - 0.4 = -0.7$	$1/(1 + e^{0.7}) = 0.332$
5	$-0.3 + 0 + 0.2 + 0.2 = 0.1$	$1/(1 + e^{-0.1}) = 0.525$
6	$(-0.3)(0.332) - (0.2)(0.525) + 0.1 = -0.105$	$1/(1 + e^{0.105}) = 0.474$

Table 10.3 Calculation of the error at each node.

Unit, j	Error, δ_j
6	$(0.474)(1 - 0.474)(1 - 0.474) = 0.1311$
5	$(0.525)(1 - 0.525)(0.1311)(-0.2) = -0.0065$
4	$(0.332)(1 - 0.332)(0.1311)(-0.3) = -0.0087$

Table 10.4 Calculations for weight and bias updating.

Weight or Bias	New Value
w_{46}	$-0.3 - (0.9)(0.1311)(0.332) = -0.339$
w_{56}	$-0.2 - (0.9)(0.1311)(0.525) = -0.262$
w_{14}	$0.2 - (0.9)(-0.0087)(1) = 0.208$
w_{15}	$-0.3 - (0.9)(-0.0065)(1) = -0.294$
w_{24}	$0.4 - (0.9)(-0.0087)(0) = 0.4$
w_{25}	$0.1 - (0.9)(-0.0065)(0) = 0.1$
w_{34}	$-0.5 - (0.9)(-0.0087)(1) = -0.492$
w_{35}	$0.2 - (0.9)(-0.0065)(1) = 0.206$
b_6	$0.1 - (0.9)(0.1311) = -0.180$
b_5	$0.2 - (0.9)(-0.0065) = 0.206$
b_4	$-0.4 - (0.9)(-0.0087) = -0.392$

network's prediction. Alternatively, we can use `SoftMax` to convert the outputs $O^i (i = 1, \dots, C)$ to the probabilities that the input tuple belongs to different class: $\frac{e^{O_i}}{\sum_{i=1}^C e^{O_i}}$. Here, O_i is the output of the i th output unit, and C is the total number of output units (e.g., the number of classes). If there is only one output node, then an output value greater than or equal to 0.5 may be considered as belonging to the positive class, while a value less than 0.5 may be considered negative.

Matrix form representation of feed-forward neural networks

For an L -layer feed-forward neural network, we can alternatively represent it in a more concise form using matrix representation. To be specific, each data tuple is represented as an n_0 -dimensional vector of attributes $\mathbf{X} = (x_1, \dots, x_{n_0})$ where n_0 is the number of attributes. Suppose each hidden or output layer has n_i units. We denote the output of units at each layer as a vector $\mathbf{h}_i$ of length n_i ($i = 0, \dots, L$), $\mathbf{h}_0 = \mathbf{X}$, and $\mathbf{h}_L$ contains the output of all output units. We denote all the weights connecting layer $(i - 1)$ to layer i as an $n_{i-1} \times n_i$ weight matrix $\mathbf{W}_i$ and all the bias values at a hidden or output layer as an n_i -

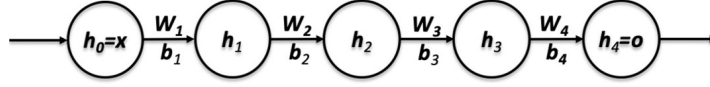


FIGURE 10.8

The matrix form representation of feed-forward neural network in Fig. 10.5. h_0 represents the input units (i.e., $h_0 = x$) and h_4 represents the output units (i.e., $h_4 = o$). For h_1, h_2, h_3 , each of them represents the hidden units at the corresponding layer.

dimensional vector b_i ($i = 1, \dots, L$). Then, each hidden or output layer essentially transforms the vector at the previous layer h_{i-1} to another vector h_i , that is, $h_i = f(W_i h_{i-1} + b_i)$, where the activation function $f(\cdot)$ operates element-wisely on a vector. What is inside the $f(\cdot)$ function ($W_i h_{i-1} + b_i$) is called an *affine transformation*.⁴ In a feed-forward neural network, each hidden or output unit is connected to *all* units in the previous layer. We call such (hidden or output) layers as *fully connected*. With such a matrix form representation, we represent an L -layer feed-forward neural network by a *chain graph* with length of L .

Example 10.2. For the four-layer neural network in Fig. 10.5, its equivalent matrix form representation is shown in Fig. 10.8, which is a chain graph of length 4. Since there are two input units in Fig. 10.5, h_0 is a vector of length 2. Likewise, h_1 is a vector of length 2 (two hidden units in the first hidden layer), h_2 is a vector of length 3 (three hidden units in the second hidden layer), h_3 is a vector of length 2 (two hidden units in the third hidden layer), and h_4 is a vector of length 1 (i.e., it is a scalar, since there is only one output unit in Fig. 10.5). The bias vector b_i has the same length as the corresponding h_i ($i = 1, 2, 3, 4$). The size of the weight matrix W_i is determined by the length of h_{i-1} (number of rows) and the length of h_i ($i = 1, 2, 3, 4$) (number of columns). For instance, since both h_0 and h_1 have length 2, the weight matrix W_1 is of size 2×2 ; since h_2 has length 3 but h_3 has length 2, the weight matrix W_2 is of size 3×2 . h_i is obtained by a nonlinear activation function $f(\cdot)$ on the affine transformation of h_{i-1} with the weight matrix W_i and the bias vector b_i ($i = 1, 2, 3, 4$). $\square$

Most deep learning models are trained in the matrix form, where a major computational bottleneck lies in matrix multiplication⁵ in both forward and backward propagation stages. In order to accelerate this process, deep learning models are often trained with GPUs (graphics processing units) instead of CPUs (central processing unit). Compared with CPUs, GPUs offer two major advantages, which are critical in training deep learning models. First, GPUs are *bandwidth optimized*, which means that GPUs are good at fetching a large amount of memory needed for matrix multiplication. Second, optimizing the bandwidth comes at the potential cost of a high memory access latency. GPUs bypass this issue (memory access latency) using a technique called *thread parallelism*.

⁴ Mathematically, an affine transformation is a linear method that changes one vector to another vector via translation, scale, shear, or rotation.

⁵ Other computationally intensive components in training deep learning models include *convolution*, the key operation in a type of deep learning model called convolutional neural networks, which will be introduced in Section 10.3.

10.1.3 Key challenges for training deep learning models

As mentioned before, the key algorithmic framework to train a deep neural network, namely the back-propagation algorithm in Fig. 10.4, has largely remained the same since the 1980s. Let us first take a closer look at the backpropagation from the optimization perspective to identify the main algorithmic challenges.

Given a set of m training tuples $\{(X^1, T^1), \dots, (X^m, T^m)\}$, where X^l and T^l are the input attribute vector and target value of the l th tuple ($l = 1, \dots, m$), respectively. We are given a feed-forward neural network with fixed architecture, including the number of layers, the number of units at each layer, and the activation function of each unit. However, we do not know the weights (w_{ij}) and bias values (b_j) of the network. Backpropagation learns such model parameters by adaptively adjusting them to minimize the (approximate) training error E

$$E(\theta) = \frac{1}{m} \sum_{l=1}^m \text{Loss}(\hat{T}(X^l, \theta), T^l), \quad (10.9)$$

where $\text{Loss}()$ is the loss function for an individual tuple, such as mean-squared loss in Fig. 10.4; θ represents all the model parameters the algorithm aims to learn, including all the weights $w_{i,j}$ and bias values b_j ; $\hat{T}(X^l, \theta)$ is the predicted target value for the l th tuple that is a (complicated) function of the model parameter θ . The overall training error $E(\theta)$ is the average loss among all m training tuples. Backpropagation starts with some initial guess of the model parameters θ_0 and then iteratively updates them as the following equation in order to minimize the training error $E(\theta)$, until the algorithm terminates,

$$\theta_{t+1} = \theta_t - \eta g_t, \quad (10.10)$$

where t is the epoch number, η is the learning rate, and g_t is the gradient of the training error $E(\theta)$ w.r.t. the model parameters θ . In the full-batch gradient descent approach, g_t is calculated by all m training tuples; in the stochastic gradient descent approach, g_t is estimated by a mini-batch of training tuples. When the algorithm terminates, we use the θ^* from the final epoch as the learned model parameters, including all the weights (w_{ij}) and the bias values b_j .

The first challenge (**optimization**) we face is how to make sure that θ^* is indeed a high-quality solution to minimize $E(\theta)$ in Eq. (10.9). If the objective function we wish to minimize is convex (e.g., the one in Fig. 10.6), then the gradient descent in Eq. (10.10) guarantees that θ^* is indeed the optimal solution regardless the initial solution θ_0 . With stochastic gradient descent, θ_t will converge to the optimal solution in the probabilistic sense. However, the training error $E(\theta)$ in a deep neural network is almost always *nonconvex*. As illustrated in Fig. 10.9, the (stochastic) gradient descent in Eq. (10.10) becomes much more challenging for a nonconvex function, even if there is only a single variable to optimize. Depending on where we start the search (i.e., the initial choice of θ_0), the algorithm might end up in a high-cost local minimal; it might be stuck in a plateau where the gradient g_t is almost 0; on the contrast the algorithm might jump over a desirable search area near a “cliff” where the gradient g_t is very large; or the algorithm might be deceived to stop at a saddle point where the gradient is zero but it neither a local minimum nor maximum. Note that in practice, due to the high complexity of the training error $E(\theta)$ of a deep neural network, it is neither realistic nor necessary to search for its global minimal. Instead, most algorithms aim to find a *high-quality local minimum*, that is, a local minimal with a low training error $E(\theta)$ in a computationally efficient way.

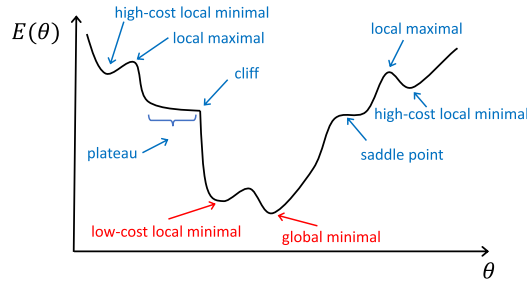


FIGURE 10.9

An illustration of the optimization challenge for training deep neural networks due to nonconvexity.

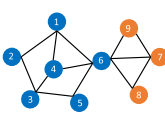
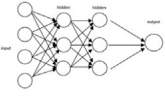
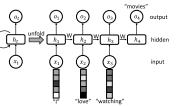
The second challenge (**generalization**) we face is overfitting. $E(\theta)$ in Eq. (10.9) is the (approximated) training error. However, what we really want is to minimize the *generalization error*, that is, the classification error on future, unseen test tuples. From what we have learned from Chapter 6, we could end up with a deep neural network model whose parameters indeed minimize the training error $E(\theta)$, but the learned network performs poorly on test tuples. Such overfitting is likely to happen, especially given the high complexity of deep neural networks with many layers and model parameters and a limited amount of labeled training tuples.

In Section 10.2, we will learn some key algorithmic techniques to address these two challenges.

10.1.4 Overview of deep learning architecture

For an L -layer feed-forward neural network, it is primarily designed for *multidimensional* data. That is, each data tuple is represented as an n_0 -dimensional vector of attributes $X = (x_1, \dots, x_{n_0})$ where n_0 is the number of attributes. In addition to feed-forward neural networks, there are many kinds of deep neural networks, which are often designed for other types of input data, including convolutional neural networks (CNNs) for *grid-like* data, recurrent neural networks (RNNs) for *sequence* data, and graph neural networks (GNNs) for *relational* (i.e., graph) data. Table 10.5 gives an overview of these

Table 10.5 An overview of typical deep learning architectures.

Data Type	Multidimensional Features: credit rating, account balance $x = (4.5, 500, 3, 5)$ #deposits, #withdrawals	Grid 	Sequence $x = \text{"I love watching movies."}$	Graph 
DL Architecture	Feed-forward Network 	CNN 	RNN 	GNN 

typical deep learning architectures. Details of these deep learning architectures will be introduced in Sections 10.3 (CNNs), 10.4 (RNNs), and 10.5 (GNNs), respectively.

10.2 Improve training of deep learning models

In this section, we introduce some key algorithmic techniques to address the two challenges (i.e., optimization and generalization) outlined in Section 10.1.3.

10.2.1 Responsive activation functions

For the backpropagation algorithm in Fig. 10.4, we have used the sigmoid function $\sigma(I)$ as the activation function, where $O = \sigma(I) = \frac{1}{1+e^{-I}} \in (0, 1)$ and its derivative is $\frac{\partial O}{\partial I} = O(1 - O)$. The error δ_j for an output unit j is calculated as $\delta_j = O_j(1 - O_j)(T_j - O_j)$, where T_j and O_j are the actual and predicted target values for a training tuple, respectively. We can see that if O_j is close to either 1 or 0, $O_j(1 - O_j)$ will be close to 0, which will in turn make the error $\delta_j \approx 0$. From Line 16 and Line 18 of Fig. 10.4, we can see that the weights increment Δw_{ij} and the bias increment Δb_j will be close to 0, and thus the weights and bias values will almost remain same in this epoch. Since $\sigma(O) \in (0, 1)$, in either case ($O_j \approx 1$ or $O_j \approx 0$), the unit is “saturated” in that its output is approaching one of its limited values. Put it in another way, when the output unit is saturated, the gradient g_t in Eq. (10.10) is close to zero, and thus (stochastic) gradient descent procedure makes very small progress in updating the model parameters θ or is even stuck (i.e., $\theta_{t+1} \approx \theta_t$). See the plateau in Fig. 10.9 for an illustration.

This issue could be further exacerbated due to the backpropagation of the error from the output layer to the hidden layers. From Line 14 of Fig. 10.4, we can see that the error δ_j of a hidden unit j might quickly approach zero due to the recursive multiplication of a set of small numbers. That is, in order to calculate the error for a hidden unit j , we first aggregate errors of units in the next higher layer that unit j connects to (i.e., $\sum_l \delta_l w_{jl}$), and then decay the aggregation result by the derivative of the sigmoid function of the unit j (i.e., $O_j(1 - O_j)$), which is always between 0 and 1. By recursively propagating the error terms backward, the error of a hidden unit, especially those from the lower layers, are likely to approach zero, even if the output unit itself is not saturated. In other words, their gradients *vanish* during the backpropagation process. Consequently, the corresponding model parameters θ (the weights and the bias values) will remain almost the same.

An effective way to address the **gradient vanishing** problem is to replace the sigmoid function with alternative, more responsive activation functions that are less likely to be saturated. **Rectified linear unit** (ReLU) is one such example. A ReLU activation function is defined as $O = f(I) = I$ if $I > 0$ and $O = f(I) = 0$ otherwise. In other words, if the net input is negative, a ReLU unit simply outputs 0 (i.e., the unit is inactive); otherwise, it just passes through the same positive net input as its output. Compared with the sigmoid function, the output of the ReLU function will be saturated only on one side when the net input is negative. The derivative of a ReLU function with respect to its net input can be calculated as $\frac{\partial O}{\partial I} = 1$ if $I > 0$ and $\frac{\partial O}{\partial I} = 0$ otherwise.⁶

⁶ If the net input $I = 0$, the derivative does not exist for the ReLU function. We can simply set it as 0. Mathematically, we can set any number between 0 and 1, known as the subgradient of the unit output w.r.t. the net input when $I = 0$.